Chapter 1 - Everyday updates and saving

Practical source and control reference
29 October 2026

Use this guide to find a feature, understand its owner, make a bounded change, and check how that change is saved. Paths are relative to C:/Dropbox/Resume Website unless stated otherwise.

Contents

Choose the right kind of edit 2

Find the feature before changing it 2

Make a small text or label change 2

Save at the correct layer 3

- Browser preferences . . . 3
- Private authored content . . . 3
- Project files and published content . . . 3

Preview without disturbing another chat 3

Coordinate source edits and recover a mistake 3

Finish an update 3

Choose the right kind of edit

Use Settings for preferences already exposed in the interface. Use Constructor for authoring controls supplied by the feature. Edit project files when changing published content, source defaults, behavior or assets. These layers are related but not interchangeable.

An example: moving Oddio's existing side panel through Constructor changes a saved configuration. Adding a fifth button to a four-button panel changes its implementation. Editing a Docs title in the browser makes a draft; Save to project deliberately writes the collection on the local review site. Publishing those project files is another action.

Find the feature before changing it

1. Open the relevant page in the local preview and identify the exact visible feature. Use Page summary to reach an About face or point of interest.
2. Open the gear. Check General first for shared behavior, then the page/face tab. Switch to Constructor only for construction/content controls.
3. If the control exists, record its old value, make one change, and check whether it survives reload in a separate review tab. Export its settings/content when available.
4. If it does not exist, use `docs/site-reference.md` to find the owner. Follow the active HTML's imports rather than choosing the similarly named script that looks newest.

Example search from PowerShell at the repository root:

```
rg -n "Document library" _6-docs
```

This finds the editor that creates the title and source controls. A visible string may be created in JavaScript or drawn on a canvas; it does not have to appear in HTML.

Make a small text or label change

1. Search for an exact, distinctive phrase in its page and shared source.
2. Decide whether the match is runtime content, a translation/label, a comment or historical evidence. Change the active runtime owner only.
3. For native HTML text, keep surrounding IDs, classes, data attributes and accessibility labels unless the task also changes their purpose.
4. For canvas/model text, follow the draw/render function. Check truncation, font size and measured hit areas after changing the label.
5. Preview normal and narrow windows. A longer label can overlap a neighboring target even when the code parses.

Do not globally rename compatible internal IDs just because you change a visible label. Fish's `about` ID and Cube's `faces` settings context are deliberate examples.

Save at the correct layer

Browser preferences

Shared settings are normally local browser state, accessed through `core/settings.js`. Existing saved values can override newly edited defaults. A fresh isolated tab/profile or a field-specific reset is more informative than clearing all storage. Never clear a user's Notes/Refs/imported-media state to diagnose a visual preference.

Private authored content

Notes drawings, Refs layouts, imported gallery images, soundfont packages, sticky-note drafts and the Docs library have private owners. Use each feature's own save/export. A full settings archive can contain references or configuration without copying large binary assets.

Before closing an edited tab, export the actual content you need. Confirm the exported file exists and, for important work, import it into a disposable review context. Keep the original open until the round trip succeeds.

Project files and published content

Save to project is an explicit local authoring operation where implemented. It is not available on every page and is not a publishing button. Sticky notes instead use a reviewed exported bundle to update `assets/annotations/sticky-notes.json`. Docs uses `documents.json` plus uploaded document/preview files. Keep both metadata and binaries when backing up a collection.

Preview without disturbing another chat

The shared local server is normally http://127.0.0.1:8878/. If it is already running, use it; do not restart it merely to refresh a page. Open a separate tab when existing tabs contain drawings or layouts.

The root `package.json` defines maintained commands. `npm run review` starts the authoring/diagnostic server. `npm run preview:artifact` is a separate static preview and does not provide the same local admin endpoints. A successful static preview therefore does not establish that a local upload workflow works.

Coordinate source edits and recover a mistake

For agent work, reserve the complete intended paths using `tools/face-file-reservations.mjs`, reread after acquisition and release after the slice. A reservation does not lock external editors. Before moving paths, coordinate active owners and obtain the human's quiet-window go-ahead where required.

Keep a feature-specific before copy or reviewed diff. To undo your edit, restore only that feature's known changes. Do not reset, clean, stash or overwrite the entire dirty repository: other chats' work can be present in the same files.

Finish an update

1. Check the edited content and its empty/error state.
2. Check pointer and keyboard access; keep typing fields from firing page shortcuts.
3. Check Reduced motion, saved values and any active audio permission.
4. Check save/export/reload for the relevant data owner.
5. Record changed paths, what was actually verified and what remains.

Remaining for this chapter: browser-check the current UI labels and add illustrated examples. File reorganization and publication are separate deferred operations.